Projektarbeit — Schritt 2: Rendering-Strategie & Kernfunktionen

Teilabgabe, unbewertet — dieser Schritt hat keine eigene Note. Die Software muss trotzdem eingecheckt werden: der Commit-Zeitstempel zählt als Nachweis für die rechtzeitige Abgabe. Die ADR hier bildet zusammen mit dem Entscheidungsprotokoll aus Schritt 3 die gemeinsame Bewertungsgrundlage.

Ziel

Ihr verlasst die Planungsphase aus Schritt 1 und beginnt die erste echte Implementierung: eine begründete Rendering-Entscheidung (SSG/CSR/SSR/ISR) pro Anwendungsteil statt eines impliziten Defaults, plus die vier zentralen Features der Plattform. Grundgerüst, Design Tokens, Architektur-ADR und State-Management-Konzept aus Schritt 1 gelten als gegeben.

Aufgaben

A. Rendering-Strategie-ADR

- Für jeden der 4 Anwendungsteile — Speaker-Profile, Session-Detailseiten, Session-Übersicht mit Filtern, „Mein Programm"-Dashboard — begründet festlegen: SSG, CSR, SSR oder ISR?
- Begründung anhand der tatsächlichen Dateneigenschaften: ändert sich der Inhalt nie zur Laufzeit (SSG)? Personalisiert, liest nur clientseitig aus `localStorage` (CSR)? Potenziell veränderlich, z. B. bei Zeitplanänderung (SSR/ISR — konzeptionelle Diskussion + Simulation reicht, kein echtes Live-Backend nötig)? Weicht ihr von der im Szenario nahegelegten Zuordnung ab: explizit begründen.
- Deliverable: ADR mit kurzer Einleitung (~100 Wörter, allgemeine Auswahlkriterien) + vier Entscheidungspunkte im Schema Kontext → Optionen [mind. 2] → Entscheidung → Konsequenzen, je 150–250 Wörter.

B. Session-Übersicht mit Filterung

- Alle 20 Sessions, filterbar nach Track, Konferenztag, Level — Filter kombinierbar (UND-Verknüpfung).
- `useConferenceData()` aus Schritt 1 tatsächlich implementieren (nicht mehr nur Konzept), Rendering-Entscheidung aus A für diesen Teil umsetzen.
- Deliverable: lauffähige, filterbare Übersichtsseite + kurzer README-Bezug zur Rendering-Entscheidung.

C. Session-Detailseiten

- Eigene Route pro Session (z. B. `/sessions/[id]`) mit Titel, Abstract, Zeit (Tag, Start-/Endzeit), Raum, Track, Level, Format, Links zu den zugehörigen Speaker-Profilen.
- Deliverable: für alle 20 Sessions korrekt befüllte, von der Übersicht erreichbare Detailseite.

D. Speaker-Profilseiten

- Eigene Route pro Speaker (z. B. `/speakers/[id]`) mit Name, Titel, Firma, Bio, Liste/Links der gehaltenen Sessions.
- Deliverable: für alle 18 Speaker korrekt befüllte, verlinkte Profilseite.

E. „Mein Programm"-Grundfunktion

- Hinzufügen/Entfernen einer Session zum persönlichen Programm, Persistenz in `localStorage` gemäß Schritt-1-Konzept (IDs vs. volle Objekte jetzt tatsächlich umsetzen), Rehydration beim Neuladen.
- Eigene „Mein Programm"-Seite, die den persistierten State anzeigt.
- Deliverable: funktionierendes Hinzufügen/Entfernen mit sichtbarer Persistenz über einen Reload hinweg, eigene Seite.

F. AI-LOG (pro Person)

- Jede Person führt ein eigenes AI-LOG über den gesamten Schritt — live mitschreiben, nicht nachträglich rekonstruieren.
- Pro Eintrag: Datum, Tool (z. B. Claude Code, Copilot, ChatGPT), Prompt im Wortlaut, Zweck (warum dieser Prompt, was sollte er lösen?).
- Deliverable: je Person ein `AI-LOG-<nachname>.md`.

Aufgabenteilung im Team

Gemeinsam:

- A (Rendering-Strategie-ADR) — betrifft die Architektur aller 4 Features, nicht einseitig festlegen.
- `useConferenceData()` zur echten Implementierung ausbauen, **bevor** die Aufteilung in B–E beginnt (alle 4 Features bauen darauf auf).
- Cross-Review vor Abgabe.

Individuell (je 1 Person federführend):

- Person 1: B (Übersicht + Filter) + D (Speaker-Profile).
- Person 2: C (Detailseiten) + E („Mein Programm").
- Verschriftlichung A: je Person 2 der 4 Entscheidungspunkte federführend, andere Person reviewt.
- F: jede Person führt ihr eigenes AI-LOG.
- Vor Abgabe: die jeweils andere Person testet die Funktion (Klick durch die Anwendung, keine offensichtlichen Brüche zwischen den 4 Features).

Abgabe

```
/docs/schritt2/
  01-rendering-strategie-adr.md
  AI-LOG-<nachname-person1>.md
  AI-LOG-<nachname-person2>.md
/src/ ...    ← implementierte Features (Übersicht+Filter, Detailseiten,
               Speaker-Profile, Mein Programm)
```

- ADR durchgängig im Schema Kontext → Optionen → Entscheidung → Konsequenzen.
- Abweichungen von der ADR durch die Implementierung (z. B. SSR-Simulation stellt sich als unpraktikabel heraus): in der ADR selbst nachtragen und begründen — nicht stillschweigend anders umsetzen.
- Commit-Historie muss den Arbeitsverlauf beider Personen erkennbar machen. README ggf. um neue Start-/Build-Befehle ergänzen.
- Abgabedeadline siehe Moodle

Bewertungsfokus (fließt in die gemeinsame Note mit Schritt 3 ein)

- **Begründungstiefe:** echte Alternativen genannt und anhand der tatsächlichen Dateneigenschaften abgewogen — nicht nur die im Szenario vorgeschlagene Zuordnung unbegründet übernommen?
- **Konsistenz ADR ↔️ Umsetzung:** entspricht die Implementierung dem, was die ADR beschreibt?
- **Funktionale Vollständigkeit:** Übersicht, Detailseiten, Speaker-Profile und „Mein Programm" sinnvoll verlinkt und für den gesamten Datensatz korrekt befüllt — auch wenn die Umsetzung noch minimal/unstyled ist.
- **Konsequente Weiterverwendung:** `useConferenceData()` als zentrale Datenschicht genutzt (nicht mehrfach parallel implementiert), `localStorage`-Persistenz funktioniert über einen Reload hinweg.
- **AI-LOG:** führt jede Person ein eigenes Log mit Datum, Tool, Prompt und begründetem Zweck pro Eintrag?